MarketTalento

Módulo 4 · Ejercicio Individual con IA · Dicampus

Ejercicio Individual de Clase

Refactorización, SRP, Streamlit y CI/CD con Python e IA

Módulo 4 · Ejercicio individual incremental · VS Code + GitHub

Nombre:	
Fecha:	
Enlace GitHub del repositorio:	

Instrucciones: Trabaja el proyecto fase a fase, un commit por bloque. Usa la IA con prompts precisos y etiqueta cada uso con [ai] en el mensaje del commit. Debes poder explicar en voz alta todo el código de tu repositorio. Revisa el código generado antes de cada push.

BLOQUE 1 — Código Heredado y Análisis de Problemas de Diseño

Recibirás varios archivos de la empresa Markettalento: InventarioAlfa.py y EndPoint_Api.py. Son dos implementaciones de una aplicación Flask monolítica que mezcla lógica de negocio, base de datos simulada, visión artificial y la interfaz HTML en un único fichero. Tu primera tarea es leerlos, ejecutarlos y documentar todos los problemas de diseño que encuentres antes de tocar una sola línea de código. Tu misión es documentarlo, testearlo y protegerlo con un pipeline CI/CD.

Característica	Detalle
Lenguaje	Python 3.9 / 3.10 / 3.11 / 3.12 (matriz CI)
Archivos heredados	InventarioAlfa.py · EndPoint_Api.py
Framework heredado	Flask + HTML embebido en render_template_string()
Framework objetivo	Streamlit 1.35+
Librerías	streamlit, pandas, flask, pytest>=8, pytest-mock>=3, flake8>=7
Commits mínimos	25 commits en convención semántica

Característica	Detalle
Duración estimada	4 – 6 horas de trabajo individual
IA permitida	Sí, con criterio. Etiquetar siempre con [ai] en el commit
Modalidad	Individual — cada alumno/a entrega su propio repositorio

1.1 — Los archivos heredados de Markettalento

Al comenzar, el repositorio contiene únicamente los dos archivos de Markettalento con los siguientes problemas conocidos:

```
# InventarioAlfa.py y EndPoint_Api.py — código heredado (NO modificar
directamente)

# PROBLEMA 1: Arquitectura monolítica
# Todo en un único fichero: BD, visión, inventario, predicción, HTML, rutas Flask

# PROBLEMA 2: HTML embebido como string Python
HTML_TEMPLATE = '''<!DOCTYPE html>...90 líneas de HTML/JS...'''
def home(): return render_template_string(HTML_TEMPLATE)

# PROBLEMA 3: Sin Principio de Responsabilidad Única (SRP)
# detect_products() mezcla: logging, lógica aleatoria y retorno de datos
# calculate_inventory_metrics() mezcla: cálculo, clasificación y recomendaciones

# PROBLEMA 4: Sin type hints
def predict_stock_outage(historial_ventas, stock_actual, producto_info):
    # ¿Qué tipo es historial_ventas? ¿lista? ¿dict? No hay forma de saberlo

# PROBLEMA 5: Sin docstrings en funciones críticas de negocio
# PROBLEMA 6: Sin tests ni pipeline CI/CD
# PROBLEMA 7: product_database hardcodeada junto a la lógica (solo 11 productos)
```

⚠ Problemas conocidos: (1) Monolito sin separación de capas. (2) HTML embebido. (3) Violación del SRP en todas las funciones. (4) Sin type hints. (5) Sin docstrings. (6) Sin tests. (7) Base de datos con solo 11 productos en 5 categorías mezcladas. Tu misión es refactorizar respetando el SRP aunque eso genere más archivos y carpetas.

BLOQUE 2 — SRP: Principio de Responsabilidad Única

Principio de Responsabilidad Única (SRP)
Cada módulo, clase o función debe tener una única razón para cambiar. Si puedes describir lo que hace una función usando la palabra "y", probablemente viola el SRP y debe dividirse.

El SRP es el primero de los principios SOLID. Aplicarlo correctamente significa que cada archivo de tu proyecto debe hacer una sola cosa bien. Es mejor tener diez archivos de 30 líneas que un único archivo de 300 líneas. Aunque parezca que hay "demasiados" archivos o carpetas, eso es correcto: la claridad y el mantenimiento lo compensan.

Ejemplos de violación del SRP en el código heredado

Función original	Responsabilidades mezcladas	Cómo dividirla
detect_products()	(1) Simular captura(2) Elegir escenario(3) Retornar datos	detector.py: solo detecciónscenario_loader.py: solo escenarios
calculate_inventory_metrics()	(1) Contar productos(2) Clasificar por estado(3) Generar recomendaciones	metrics.py: solo métricasrecommender.py: solo recomendaciones
predict_stock_outage()	(1) Calcular media de ventas(2) Predecir días(3) Asignar estado	stock_predictor.py: solo predicciónde demand_analyzer.py: solo análisis de demanda
calculate_inventory_value()	(1) Buscar precio en BD(2) Calcular multiplicación(3) Sumar total	valuation.py: una función, un cálculo

Regla práctica: Si al describir una función dices "calcula la media Y clasifica el estado Y genera la recomendación", contiene tres responsabilidades. Divídela en tres funciones, en tres archivos si es necesario. El exceso de archivos es un síntoma de buena arquitectura, no de un problema.

BLOQUE 3 — Modularización: Arquitectura y Estructura

3.1 — Arquitectura del Proyecto (resultado final)

Esta es la estructura de archivos que debe tener tu repositorio al entregar. Respeta el SRP: cada archivo tiene una única responsabilidad, aunque eso implique más carpetas que en el código original.

```
markettalento-inventario/
+-- services/                        # Toda la lógica de negocio
|   +-- __init__.py
|   +-- database/                    # Responsabilidad: datos
|       +-- __init__.py
|       +-- product_db.py            # Solo los datos: los 25 productos
|       +-- db_reader.py             # Solo lectura: get_product_info(),
get_all_products()
|       +-- db_filter.py             # Solo filtrado: get_by_category(),
get_sales_history()
|   +-- vision/                      # Responsabilidad: detección de imagen
|       +-- __init__.py
|       +-- detector.py              # Solo orquestación: detect_products()
|       +-- scenario_loader.py       # Solo escenarios: _load_scenarios()
|   +-- inventory/                   # Responsabilidad: análisis de inventario
|       +-- __init__.py
|       +-- metrics.py              # Solo métricas: count_by_status()
|       +-- valuation.py            # Solo valoración: calculate_inventory_value()
|       +-- recommender.py          # Solo recomendaciones:
generate_recommendations()
|   +-- prediction/                  # Responsabilidad: predicción de demanda
|       +-- __init__.py
|       +-- stock_predictor.py      # Solo predicción: predict_stock_outage()
|       +-- demand_analyzer.py      # Solo análisis: calculate_daily_demand()
+-- interface/                       # Responsabilidad: presentación
|   +-- demoStreamlit.py            # Solo la UI Streamlit
+-- tests/                           # Responsabilidad: verificación
|   +-- __init__.py
|   +-- services/
|       +-- database/
|           +-- test_product_db.py
|           +-- test_db_reader.py
|           +-- test_db_filter.py
|       +-- vision/
|           +-- test_detector.py
|           +-- test_scenario_loader.py
|       +-- inventory/
|           +-- test_metrics.py
|           +-- test_valuation.py
|           +-- test_recommender.py
|       +-- prediction/
|           +-- test_stock_predictor.py
```

```
|           +-- test_demand_analyzer.py
|   +-- integration/
|       +-- test_full_pipeline.py
+-- docs/
|   +-- arquitectura.md             # Diagrama y descripción de módulos
|   +-- Final.md                    # Memoria de cambios realizados
+-- .github/
|   +-- workflows/
|       +-- ci.yml                  # Matriz: Python 3.9 / 3.10 / 3.11 / 3.12
+-- requirements.txt
+-- .gitignore                      # __pycache__, .venv, *.pyc
+-- README.md                       # Instalación, uso, sección IA, badge CI
```

3.2 — Plan de 16 Commits en Orden Estricto

FASE 1 — Estructura inicial

#	Commit	Qué debe contener	Tarea
01	chore: init repo y .gitignore	Repo en GitHub. .gitignore con __pycache__, .venv, *.pyc	Alumno/a
02	chore: estructura de carpetas SRP [ai]	Carpetas services/, interface/, tests/ con subcarpetas y __init__.py. requirements.txt	Alumno/a

FASE 2 — Capa de datos (services/database/)

#	Commit	Qué debe contener	Tarea
03	feat: product_db.py — 25 productos [ai]	product_database con los 25 productos en 5 categorías (Refrigerados, Conservas, Bebidas, Panadería, Despensa). Type hints en el dict	Alumno/a
04	feat: db_reader.py y db_filter.py [ai]	get_product_info(), get_all_products() en db_reader.py. get_by_category(), get_sales_history() en db_filter.py. Docstrings Google en todas	Alumno/a

FASE 3 — Capa de servicios (vision, inventory, prediction)

#	Commit	Qué debe contener	Tarea
05	feat: scenario_loader.py y detector.py [ai]	_load_scenarios() en scenario_loader.py (solo datos). detect_products() en detector.py (solo orquestación). Type hints. Docstrings	Alumno/a
06	feat: metrics.py y valuation.py [ai]	count_by_status() en metrics.py. calculate_inventory_value() en valuation.py. Cada función con una única responsabilidad	Alumno/a
07	feat: recommender.py [ai]	generate_recommendations() como función independiente. Docstring Google completo con Args, Returns, Example	Alumno/a

#	Commit	Qué debe contener	Tarea
08	feat: demand_analyzer.py y stock_predictor.py [ai]	calculate_daily_demand() en demand_analyzer.py. predict_stock_outage() en stock_predictor.py. Maneja historial vacío con ValueError	Alumno/a

FASE 4 — Tests unitarios

#	Commit	Qué debe contener	Tarea
09	test: tests de database/ [ai]	test_product_db.py, test_db_reader.py, test_db_filter.py. Cubrir valores válidos, inválidos, lista vacía, filtro por categoría	Alumno/a
10	test: tests de vision/ [ai]	test_detector.py, test_scenario_loader.py. Mock de random.choice(). Verificar estructura del escenario. Edge case: lista vacía	Alumno/a
11	test: tests de inventory/ [ai]	test_metrics.py, test_valuation.py, test_recommender.py. Cubrir estado crítico, bajo, normal. Lista vacía. Producto no encontrado en BD	Alumno/a
12	test: tests de prediction/ [ai]	test_demand_analyzer.py, test_stock_predictor.py. pytest-mock para historial de ventas. Verificar ValueError con historial vacío	Alumno/a
13	test: test_full_pipeline.py — integración [ai]	Test de integración end-to-end: detect_products → predict → recommend. Verifica que la cadena completa no lanza excepciones	Alumno/a

FASE 5 — Pipeline CI/CD

#	Commit	Qué debe contener	Tarea
14	feat: ci.yml con matriz Python [ai]	Pipeline con strategy.matrix en Python 3.9, 3.10, 3.11 y 3.12. Pasos: checkout, setup-python, pip install, flake8, pytest -v	Alumno/a
15	test: verificar pipeline en GitHub Actions	Push. Captura del pipeline en verde (4 versiones) incluida en README. Corregir cualquier fallo de linting	Alumno/a

FASE 6 — Streamlit y cierre

#	Commit	Qué debe contener	Tarea
16	feat: demoStreamlit.py + docs + README [ai]	interface/demoStreamlit.py con las 3 secciones. docs/Final.md completado. README con instalación, ejecución Streamlit, badge CI y captura	Alumno/a

BLOQUE 4 — Base de Datos Ampliada: 25 Productos

La empresa Markettalento gestiona 25 productos organizados en 5 categorías. El archivo services/database/product_db.py debe contener exactamente esta base de datos. Está estructurada como un dict de Python con type hints explícitos en la declaración.

4.1 — Categoría: Refrigerados (frío)

ID	Nombre	Precio €	Unidad	Stk.Min	Stk.Max	T.Repos (días)
1	Leche Entera	1.20	litro	5	30	2
2	Yogur Natural	0.55	unidad	8	60	2
3	Queso Fresco	2.80	pieza	4	20	3
4	Mantequilla	1.95	tarrina	4	25	4
5	Fiambre Pavo	3.50	paquete	3	18	2
6	Crema de Leche	0.90	brik	5	35	3
7	Huevos M	2.50	docena	3	20	1

4.2 — Categoría: Conservas

ID	Nombre	Precio €	Unidad	Stk.Min	Stk.Max	T.Repos (días)
8	Atún en Aceite	1.60	lata	10	80	7
9	Tomate Triturado	0.75	bote	12	100	7
10	Judías Blancas	1.10	bote	8	60	7
11	Aceitunas Verdes	1.40	bote	6	50	10
12	Sardinas en Aceite	1.80	lata	8	60	7
13	Maíz Dulce	0.95	lata	6	50	7

4.3 — Categoría: Bebidas

ID	Nombre	Precio €	Unidad	Stk.Min	Stk.Max	T.Repos (días)
14	Agua Mineral	0.60	botella	15	100	3
15	Café Molido	4.50	paquete	3	25	5
16	Zumo de Naranja UHT	1.35	brik	8	60	5
17	Refresco Cola	1.20	lata	10	80	4
18	Cerveza 1905	1.10	botella	6	50	5

4.4 — Categoría: Panadería

ID	Nombre	Precio €	Unidad	Stk.Min	Stk.Max	T.Repos (días)
19	Pan de Molde	1.40	paquete	10	50	1
20	Baguette	0.90	unidad	12	60	1
21	Croissant	0.75	unidad	8	40	1

4.5 — Categoría: Despensa

ID	Nombre	Precio €	Unidad	Stk.Min	Stk.Max	T.Repos (días)
22	Arroz	1.80	kg	10	60	4
23	Aceite de Oliva	4.20	litro	8	40	4
24	Azúcar	1.50	kg	10	50	3
25	Harina de Trigo	1.20	kg	10	60	3

> Nota: El historial de ventas de cada producto debe ser una lista de 20 enteros representando las unidades vendidas por día. Los valores deben ser coherentes con la categoría: los Refrigerados tienen alta rotación diaria, las Conservas menor rotación. Define los valores tú mismo con criterio y documenta el criterio elegido en docs/Final.md.

BLOQUE 5 — Tests Unitarios con pytest

La suite de tests refleja la estructura SRP del código: cada archivo de test cubre exactamente un módulo. Los tests deben seguir la convención AAA (Arrange-Act-Assert) y tener docstring Google en cada función de test.

5.1 — Mapa de cobertura requerida

Archivo de test	Módulo que cubre	Casos mínimos requeridos
test_product_db.py	product_db.py	25 productos presentes, categorías correctas, tipos de datos
test_db_reader.py	db_reader.py	get_product_info() válido, inválido, None
test_db_filter.py	db_filter.py	get_by_category() con categoría existente, inexistente, vacía
test_scenario_loader.py	scenario_loader.py	Escenarios tienen lista 'productos', cada producto tiene 'nombre' y 'cantidad'
test_detector.py	detector.py	Mock de random.choice(). Resultado tiene clave 'productos'. Lista nunca vacía
test_metrics.py	metrics.py	Estado CRÍTICO, BAJO, NORMAL. Lista vacía devuelve zeros
test_valuation.py	valuation.py	Valor correcto (precio × stock), producto sin BD devuelve 0, lista vacía = 0.0
test_recommender.py	recommender.py	Lista crítica genera recomendación urgente, lista normal genera OK
test_demand_analyzer.py	demand_analyzer.py	Media correcta, historial vacío lanza ValueError, historial de un elemento
test_stock_predictor.py	stock_predictor.py	Mock de historial, días correctos, estado correcto por rango
test_full_pipeline.py	Integración completa	detect → predict → recommend no lanza excepciones, resultado tiene claves esperadas

5.2 — Formato obligatorio de docstring en tests

```
def test_calculate_daily_demand_con_historial_normal():
    """Verifica que la demanda diaria se calcula correctamente.

    Args: Ninguno.
```

```
    Returns: None.
    Example:
        >>> result = calculate_daily_demand([3, 4, 5])
        >>> result == 4.0
        True
    """
    # Arrange
    historial = [3, 4, 5, 4, 3, 5, 4, 3, 4, 5]
    # Act
    resultado = calculate_daily_demand(historial)
    # Assert
    assert resultado == 4.0
```

Regla de oro: Si no tienes docstring en el test, el commit no es válido. Usa la IA para generar el esqueleto y completa los ejemplos tú mismo/a.

BLOQUE 6 — Pipeline CI/CD con Matriz de Versiones Python

El pipeline de Markettalento debe garantizar que el código funciona en Python 3.9, 3.10, 3.11 y 3.12. Para ello se usa la directiva strategy.matrix de GitHub Actions, que ejecuta el mismo job en paralelo con cada versión.

```
name: Markettalento InventarioAlfa CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  quality:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: ["3.9", "3.10", "3.11", "3.12"]
      fail-fast: false       # Ejecuta todas aunque una falle

    steps:
```

```
      - uses: actions/checkout@v4

      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Lint with flake8
        run: flake8 services/ interface/ tests/ --max-line-length=100

      - name: Run tests with pytest
        run: pytest tests/ -v --tb=short
```

Importante: Con la matriz activada, GitHub Actions lanzará 4 ejecuciones en paralelo (una por versión Python). En la pestaña Actions verás 4 checks verdes. El badge en el README debe mostrar el estado de la rama main. El pipeline debe estar en verde en las 4 versiones antes del commit 16.

6.1 — Reflexión sobre la matriz de versiones

En docs/Final.md documenta:

- ¿Hubo alguna versión Python donde los tests fallaron y en otras no? ¿Por qué?
- ¿Qué dependencia del requirements.txt causó problemas con alguna versión?
- ¿Cuál es la ventaja de usar fail-fast: false en un equipo de desarrollo?

BLOQUE 7 — Interfaz Streamlit: interface/demoStreamlit.py

El archivo interface/demoStreamlit.py es la única interfaz de usuario del proyecto. Reemplaza completamente el HTML embebido de Flask. Importa exclusivamente de services/ y no contiene lógica de negocio.

7.1 — Secciones obligatorias del dashboard

Sección A — Resultados del Análisis (métricas con st.metric)

Productos Detectados	Unidades Totales	Productos Críticos	Stock Bajo
5	34	0	2

Sección B — Detalle del Análisis (tabla con st.dataframe)

Producto	Stock Actual	Estado	Días hasta Agotarse	Recomendación
Leche Entera	8	Normal	2	Monitorear
Huevos M	2	Bajo	0.7	Reponer urgente
Pan de Molde	3	Crítico	0.3	Reponer YA
Agua Mineral	12	Normal	1.1	OK
Café Molido	6	Normal	4.0	Monitorear

Sección C — Recomendaciones de Reposición y Valor del Inventario

- 🚨 Recomendaciones de Reposición: lista de productos con estado Crítico o Bajo, con mensaje de acción claro
- 💰 Valor del Inventario: suma de (stock × precio) para todos los productos detectados, formateado con 2 decimales y símbolo €

7.2 — Fragmento de referencia Streamlit

```
import streamlit as st
import pandas as pd
from services.database.product_db import product_database
from services.database.db_reader import get_all_products
from services.vision.detector import detect_products
from services.inventory.metrics import count_by_status
from services.inventory.valuation import calculate_inventory_value
from services.inventory.recommender import generate_recommendations
from services.prediction.stock_predictor import predict_stock_outage

st.set_page_config(page_title='Markettalento Inventario', page_icon='📦',
layout='wide')
st.title('📦 Sistema de Inventario — Markettalento')
```

```
if st.button('🔍 Analizar Inventario'):
    deteccion = detect_products()
    productos = deteccion['productos']

    # Sección A — Métricas
    metricas = count_by_status(productos, product_database)
    col1, col2, col3, col4 = st.columns(4)
    col1.metric('Productos Detectados', metricas['total'])
    col2.metric('Unidades Totales', metricas['total_unidades'])
    col3.metric('Productos Críticos', metricas['criticos'])
    col4.metric('Stock Bajo', metricas['bajos'])

    # Sección B — Tabla de detalle
    st.subheader('📊 Detalle del Análisis')
    filas = []
    for p in productos:
        pred = predict_stock_outage(p, product_database)
        rec = generate_recommendations([pred]) if pred['estado'] != 'Normal' else
'OK'
        filas.append({...})
    st.dataframe(pd.DataFrame(filas), use_container_width=True)

    # Sección C — Recomendaciones y valor
    st.subheader('🚨 Recomendaciones de Reposición')
    # ... mostrar productos que necesitan reposición
    st.subheader('💰 Valor del Inventario')
    valor = calculate_inventory_value(productos, product_database)
    st.metric('Valor Total', f'{valor:.2f} €')
```

SRP en la interfaz: demoStreamlit.py no debe contener ninguna lógica de negocio. Si copias una fórmula de cálculo dentro de demoStreamlit.py, estás violando el SRP. Toda la lógica vive en services/. La interfaz solo llama funciones y muestra resultados.

BLOQUE 8 — Documentación y Memoria Final.md

8.1 — Estructura de docs/

Archivo	Contenido requerido
docs/arquitectura.md	Diagrama ASCII de la arquitectura original (monolito) vs. modular SRP. Descripción de cada módulo y su única responsabilidad
docs/Final.md	Memoria completa: qué estaba mal, qué se cambió, por qué, tabla de uso de IA, y reflexión sobre la matriz de versiones

8.2 — Preguntas de la Memoria Final.md

8.2.1 — ¿Por qué el código original viola el SRP?

8.2.2 — Tabla de módulos y responsabilidades

Módulo	Única responsabilidad asignada	Por qué no se mezcla con otro
product_db.py		
db_reader.py		
detector.py		
recommender.py		
stock_predictor.py		
demoStreamlit.py		

8.2.3 — Inventario de uso de IA

Archivo	Prompt principal que usaste	Cambios que hiciste tú
prediction/stock_predictor.py		
tests/prediction/test_stock_predictor.py		
interface/demoStreamlit.py		
.github/workflows/ci.yml		

8.2.4 — ¿Qué fallo ocurrió en la matriz de versiones Python?

8.2.5 — ¿Cuándo decidiste NO usar la IA y por qué?

BLOQUE 9 — Reglas de Uso de IA

La IA es tu copiloto, no tu piloto. Etiqueta con [ai] cada commit donde participó. Debes poder explicar en voz alta todo el código de tu repositorio.

SÍ puedes hacer	NO puedes hacer
✔ Pedir a la IA que identifique violaciones del SRP ✔ Generar el esqueleto de docstrings y completar los ejemplos tú ✔ Pedir casos base de tests y añadir edge cases propios ✔ Generar el ci.yml con la matriz de versiones ✔ Pedir a la IA que divida una función en dos con SRP ✔ Pedir que explique el código generado	✖ Copiar código sin leerlo ni entenderlo ✖ Pedir 'hazme toda la refactorización de una vez' ✖ No poner [ai] cuando la IA ayudó ✖ Aceptar tests sin verificar que pasan en verde ✖ Poner lógica de negocio en demoStreamlit.py ✖ No saber explicar el SRP si el profesor pregunta

Plantilla de prompt SRP recomendada

```
CONTEXTO: Proyecto Markettalento en Python 3.10, con SRP estricto.
ARCHIVO: services/inventory/metrics.py
FUNCIÓN A SEPARAR: calculate_inventory_metrics() del código heredado.
NECESITO: Extraer SOLO la parte que cuenta y clasifica productos por estado.
          La función debe llamarse count_by_status(detected: list, db: dict) ->
dict
          y devolver {'total': int, 'criticos': int, 'bajos': int, 'normales':
int}.
REQUISITOS: type hints, docstring Google (Args, Returns, Raises, Example),
            NO incluir generate_recommendations() — esa va en recommender.py.
NO incluyas imports duplicados ni código fuera de la función.
```

BLOQUE 10 — Retos Opcionales — para nota extra

Reto	Descripción
⭐ Coverage Report	Añade pytest-cov al pipeline. Badge de cobertura en README. Objetivo: >= 80% de cobertura en services/
⭐ Filtro por categoría en Streamlit	Añade un selectbox en demoStreamlit.py para filtrar la tabla por categoría (Refrigerados, Conservas, Bebidas…)
⭐ Gráfico de barras Streamlit	Sección extra con st.bar_chart mostrando el historial de ventas del producto seleccionado
⭐⭐ Patrón Repository	Refactoriza db_reader.py a una clase ProductRepository con get_all(), get_by_name(), get_by_category(). Actualiza todos los tests
⭐⭐ Pre-commit hooks	Configura pre-commit con flake8 y black. El README explica la instalación. Commits rechazados si linter falla
⭐⭐ API REST independiente	Mantén EndPoint_Api.py como microservicio Flask separado y consúmelo desde demoStreamlit.py con requests.get()